Secure Authentication System

A Beginner-Friendly Flask Backend with SQLite, Bcrypt & JWT

Presented By: Ankit Kumar Singh

Course Tracker: B.Tech Computer Science Engineering (CSE)

Academic Batch: 2025-2029

Project Workspace Directory Tree

The application workspace is structured following professional, isolated folder tracking matrices:

```
■ secure authentication/
■■■ ■ static/
■   ■■■ ■ script.js
■   ■■■ ■ style.css
■■■ ■ templates/
■   ■■■ ■ index.html      ← Consolidated user login, register, and workspace interface
■■■ ■ .gitignore          ← Explicitly excludes venv/, database files, and system caches
■■■ ■ app.py              ← The main Python web framework engine
■■■ ■ README.md           ← Complete setup documentation and API specification guide
■■■ ■ requirements.txt    ← Python package configuration ledger
■■■ ■ users.db            ← Local SQLite database instance generated at initial run
```

Core Backend Architecture Stack

The system integrates industrial security primitives decoupled into accessible modules:

• **Python Flask:** Serves lightweight routing, request mapping, and template layout rendering pipelines.

• **SQLite Engine:** Lightweight database system mapping relational user parameters without high infrastructure costs.

• **Bcrypt Library:** Secure hashing functions that utilize structural cryptographic salting to prevent exposure.

• **PyJWT Framework:** Issues compact, stateless tokens that automatically manage active sessions without database bottlenecks.

Cryptographic Salting Engine via Bcrypt

Plaintext passwords are vulnerable to leakage. This platform ensures zero plaintext persistence:

```
# Password processing mechanism during new user account registration
hashed_password = bcrypt.generate_password_hash(password).decode('utf-8')

# High-speed credential validation routine during user authentication matching
is_valid = bcrypt.check_password_hash(user['password'], input_password)
```

Security Impact: Even if the database `users.db` is compromised, the original user passwords remain safe because they are converted into irreversible, cryptographically secure hash strings.

Stateless JWT Authentication Pipeline

Traditional sessions rely heavily on persistent server-side storage states. This implementation relies on signed JWT claims:

```
token_payload = {
    'user_id': user['id'],
    'username': user['username'],
    'exp': datetime.datetime.utcnow() + datetime.timedelta(hours=2) # 2-Hour Window
}
token = jwt.encode(token_payload, app.config['SECRET_KEY'], algorithm='HS256')
```

- **Signature Validation:** Protected endpoints demand an `Authorization: Bearer <token>` header frame.
- **Timeout Guardrails:** Tokens automatically expire exactly 2 hours after issuance, protecting user sessions.

REST API Architectural Endpoints

The backend engine provides clear, structured endpoints using appropriate HTTP status codes:

Endpoint Route	HTTP Method	Functional Responsibility Matrix
`/api/register`	POST	Validates fields, salts passwords, and inserts new record lines.
`/api/login`	POST	Matches hashes and returns signed 2-hour JWT credentials.
`/api/profile`	GET	Parses authentication headers and decodes secure parameters.

Environment Security Configurations

• **Decoupled Sensitive Keys:** All cryptographic keys are loaded using environment variables (`.env`) via `python-dotenv`, keeping them out of source control.

• **Source Control Isolation:** The `.gitignore` file keeps developer environments secure by blocking uploads of `venv/`, `.env`, hidden data layers, and compiled `__pycache__/` blocks.

• **Robust Error Resilience:** Custom `try-except` catch blocks handle database issues (like duplicate usernames or expired tokens) gracefully, returning clean JSON error responses.

Project Submission Verification Links

The project repository workspace has been restructured and is ready for academic review.

■ **Source Code Repository:**
https://github.com/ankitsingh085432-afk/2025-2029_Ankitkumarsingh_4110_3sme_2cse13-

■ **Active Sentiment Web Link:**
https://two025-2029-ankitkumarsingh-4110-3sme-h1xn.onrender.com

Thank You! Open for Questions and Verification.